

СОПРОВОДИТЕЛЬНАЯ ЗАПИСКА К ПРОЕКТУ

Выполнил студент Ильин Константин

16.03.2026г.

Пермь, 2026

ОГЛАВЛЕНИЕ

1. СХЕМА БАЗЫ ДАННЫХ.....	3
2. ДИАГРАММА КЛАССОВ	4
3. ПРОЦЕСС СОЗДАНИЯ РОЛИ, ТАБЛИЧНОГО ПРОСТРАНСТВА, БАЗЫ ДАННЫХ И СХЕМЫ	5
4. ПРОЦЕСС СОЗДАНИЯ ПРИЛОЖЕНИЯ ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ POSTGRESQL	10
4.1 Создание проекта в Microsoft Visual Studio	10
4.2 Написание классов в библиотеке классов.....	11
4.3 Реализация бизнес-логики приложения.....	15
4.4 Написание модульных тестов.....	24
4.5 Реализация пользовательского интерфейса приложения	30
5. ВЫВОД	39

1. СХЕМА БАЗЫ ДАННЫХ

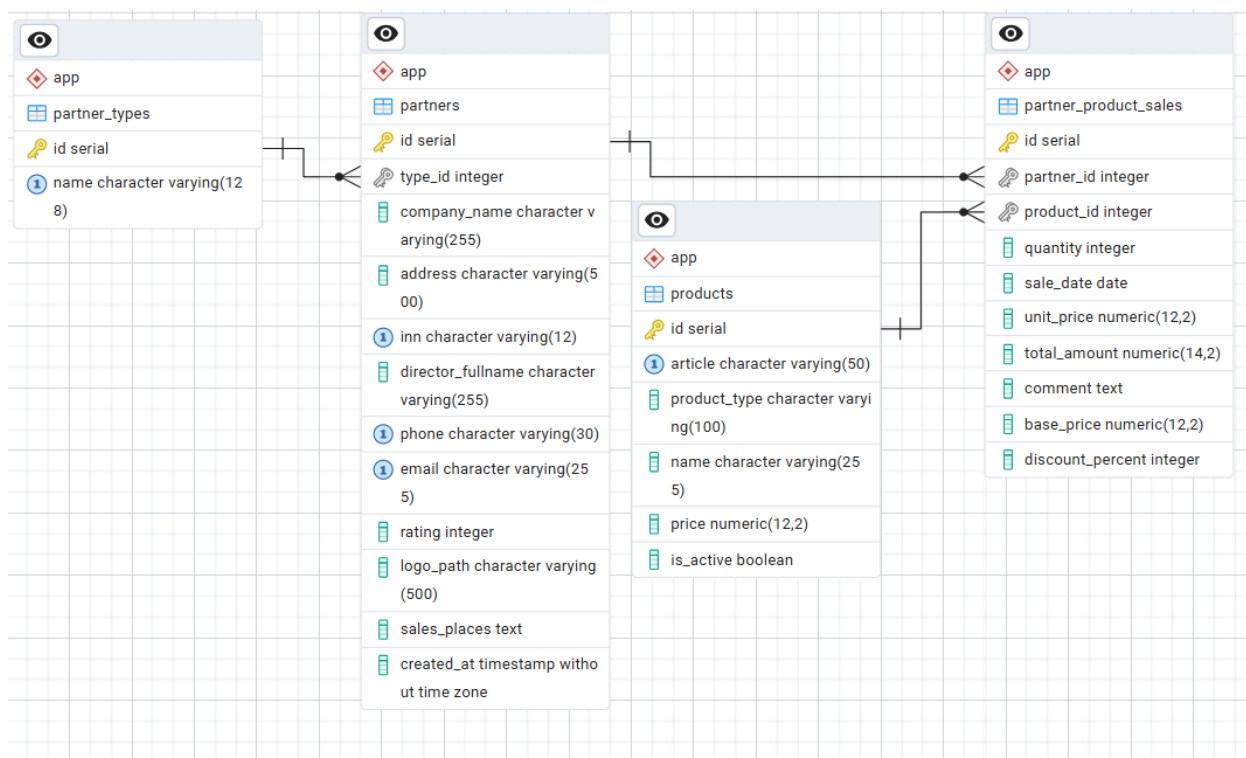


Рисунок 1. Схема базы данных

2. ДИАГРАММА КЛАССОВ

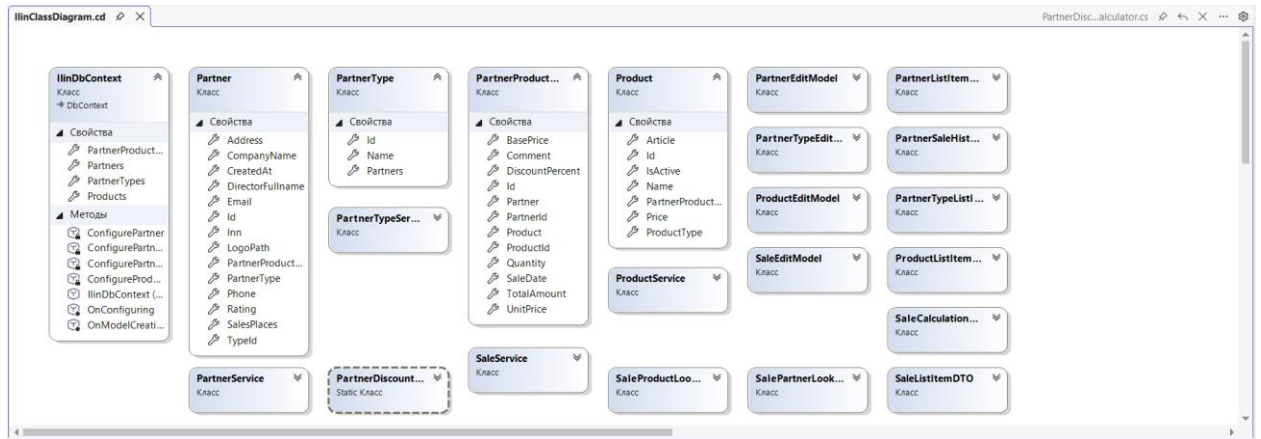


Рисунок 2. Диаграмма классов

3. ПРОЦЕСС СОЗДАНИЯ РОЛИ, ТАБЛИЧНОГО ПРОСТРАНСТВА, БАЗЫ ДАННЫХ И СХЕМЫ

Перед началом разработки приложения была выполнена подготовка среды хранения данных. В качестве системы управления базами данных была выбрана PostgreSQL, поскольку данная СУБД является надежной, широко распространенной и поддерживает все необходимые механизмы работы с реляционными данными.

Для обеспечения безопасного доступа приложения к базе данных было принято решение создать отдельную роль пользователя, которая будет использоваться программой при подключении к базе данных.

Сначала был выполнен вход в консоль управления PostgreSQL psql под учетной записью администратора базы данных. После этого была создана отдельная роль для приложения. Роль предназначена для выполнения операций чтения и записи данных в базе данных.

Команда создания роли имеет следующий вид:

```
CREATE ROLE app LOGIN PASSWORD '123456789';
```

Данная команда создает пользователя с именем **app**, который имеет возможность выполнять подключение к серверу PostgreSQL.

После создания роли была создана отдельная база данных, в которой будут храниться данные разрабатываемого приложения.

Создание базы данных выполняется следующей командой:

```
CREATE DATABASE ilin;
```

После создания базы данных было выполнено подключение к ней:

```
\c ilin
```

Далее внутри базы данных была создана отдельная схема **app**, предназначенная для хранения всех таблиц приложения. Использование

отдельной схемы позволяет логически отделить структуру данных приложения от системных объектов базы данных.

Создание схемы выполняется следующей командой:

```
CREATE SCHEMA app;
```

После создания схемы необходимо выдать роли приложения права доступа на работу с объектами данной схемы. Для этого были выполнены следующие команды:

```
GRANT USAGE, CREATE ON SCHEMA app TO app;
```

Данная команда предоставляет пользователю **app** возможность использовать схему и создавать в ней объекты базы данных.

Далее были выданы права на работу со всеми таблицами, которые будут создаваться внутри схемы:

```
GRANT SELECT, INSERT, UPDATE, DELETE  
ON ALL TABLES IN SCHEMA app  
TO app;
```

Эта команда разрешает приложению выполнять основные операции над данными: чтение, добавление, изменение и удаление записей.

Также были выданы права на использование последовательностей, которые применяются для генерации идентификаторов записей:

```
GRANT USAGE, SELECT  
ON ALL SEQUENCES IN SCHEMA app  
TO app;
```

Для того чтобы новые таблицы и последовательности автоматически получали необходимые права доступа, были настроены **права по умолчанию**. Это позволяет избежать необходимости вручную назначать права при создании новых объектов базы данных.

Настройка прав по умолчанию выполняется следующими командами:

```
ALTER DEFAULT PRIVILEGES IN SCHEMA app
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app;
```

```
ALTER DEFAULT PRIVILEGES IN SCHEMA app
GRANT USAGE, SELECT ON SEQUENCES TO app;
```

После выполнения данных действий была подготовлена полноценная среда хранения данных для работы приложения. Созданная база данных и схема позволяют безопасно хранить информацию о партнерах, товарах и продажах, а отдельная роль приложения обеспечивает контролируемый доступ к данным со стороны программного обеспечения.

Ниже предоставлены SQL-кода на создание таблиц, а также индексы.

```
CREATE TABLE partners
(
    id                SERIAL PRIMARY KEY,
    type_id           INTEGER NOT NULL,
    company_name      VARCHAR(255) NOT NULL,
    address           VARCHAR(500) NOT NULL,
    inn               VARCHAR(12) UNIQUE CHECK (inn IS NULL OR
char_length(inn) IN (10, 12)),
    director_fullname VARCHAR(255) NOT NULL,
    phone             VARCHAR(30) NOT NULL UNIQUE,
    email             VARCHAR(255) NOT NULL UNIQUE,
    rating            INTEGER NOT NULL DEFAULT 0 CHECK (rating
>= 0),
```

```

        logo_path          VARCHAR(500),
        sales_places       TEXT,
        created_at         TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
        FOREIGN KEY (type_id) REFERENCES partner_types(id) ON UPDATE
CASCADE ON DELETE RESTRICT
    );
CREATE TABLE products
(
    id                     SERIAL PRIMARY KEY,
    article               VARCHAR(50) NOT NULL UNIQUE,
    product_type          VARCHAR(100),
    name                  VARCHAR(255) NOT NULL,
    price                 NUMERIC(12, 2) NOT NULL DEFAULT 0 CHECK
(price >= 0),
    is_active             BOOLEAN NOT NULL DEFAULT TRUE
);
CREATE TABLE partner_product_sales
(
    id                     SERIAL PRIMARY KEY,
    partner_id            INTEGER NOT NULL,
    product_id            INTEGER NOT NULL,
    quantity              INTEGER NOT NULL CHECK (quantity > 0),
    sale_date             DATE NOT NULL,
    unit_price            NUMERIC(12, 2) NOT NULL CHECK (unit_price
>= 0),
    total_amount          NUMERIC(14, 2) GENERATED ALWAYS AS
(quantity * unit_price) STORED,
    comment               TEXT,
    base_price            NUMERIC(12,2) NOT NULL DEFAULT 0,
    discount_percent      INT NOT NULL DEFAULT 0,
    FOREIGN KEY (partner_id) REFERENCES partners(id) ON UPDATE CASCADE
ON DELETE CASCADE,

```



```

        FOREIGN KEY (product_id) REFERENCES products(id) ON UPDATE CASCADE
ON DELETE RESTRICT
);

CREATE INDEX IF NOT EXISTS idx_partners_partner_type_id
    ON partners(type_id);

CREATE INDEX IF NOT EXISTS idx_partners_company_name
    ON partners(company_name);

CREATE INDEX IF NOT EXISTS idx_sales_partner_id
    ON partner_product_sales(partner_id);

CREATE INDEX IF NOT EXISTS idx_sales_product_id
    ON partner_product_sales(product_id);

CREATE INDEX IF NOT EXISTS idx_sales_sale_date
    ON partner_product_sales(sale_date);

CREATE INDEX IF NOT EXISTS idx_sales_partner_date
    ON partner_product_sales(partner_id, sale_date);

```

```

Командная строка - psql -U | x
Microsoft Windows [Version 10.0.26200.8037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\lstr5>chcp 1251
Текущая кодовая страница: 1251

C:\Users\lstr5>psql -U postgres
Пароль пользователя postgres:

psql (18.3)
Введите "help", чтобы получить справку.

postgres=# \c ilin
Вы подключены к базе данных "ilin" как пользователь "postgres".
ilin=# set search_path to app;
SET
ilin=# \dt

```

Схема	Имя	Тип	Владелец
app	partner_product_sales	таблица	postgres
app	partner_types	таблица	postgres
app	partners	таблица	postgres
app	products	таблица	postgres

```

(4 строки)

ilin=#

```

Рисунок 3. Список созданных таблиц

4. ПРОЦЕСС СОЗДАНИЯ ПРИЛОЖЕНИЯ ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ POSTGRESQL

После подготовки базы данных был выполнен процесс разработки программного приложения, предназначенного для работы с созданной базой данных. Приложение реализовано на платформе **.NET** с использованием технологии **WPF (Windows Presentation Foundation)** для создания графического пользовательского интерфейса.

Основной целью разработки приложения является автоматизация учета партнеров, товаров и истории продаж. Приложение позволяет хранить данные о партнерах организации, регистрировать продажи товаров и автоматически рассчитывать скидки для партнеров на основе объема сотрудничества.

Для удобства разработки и разделения логики система была разделена на несколько проектов:

- библиотека классов, содержащая модели данных и бизнес логику;
- WPF-приложение, реализующее пользовательский интерфейс;
- тестовый проект для выполнения модульных тестов.

4.1 Создание проекта в Microsoft Visual Studio

Разработка программного продукта выполнялась в среде разработки **Microsoft Visual Studio**. В рамках решения были созданы несколько проектов, объединенных в одно решение.

Первым был создан проект типа **Class Library**, который содержит основные классы системы, модели данных, сервисы и контекст базы данных. Данный проект выполняет роль слоя доступа к данным и бизнес-логики приложения.

Далее был создан проект типа **WPF Application**, предназначенный для реализации пользовательского интерфейса программы. В этом проекте размещены окна приложения, элементы управления и логика взаимодействия пользователя с системой.

Также был создан отдельный проект для выполнения **модульного тестирования**, реализованный с использованием фреймворка **MSTest**. Данный проект используется для проверки корректности работы ключевых методов бизнес-логики приложения.

Такая архитектура позволяет разделить пользовательский интерфейс, бизнес-логику и тестирование, что повышает удобство разработки и сопровождения программного продукта.

4.2 Написание классов в библиотеке классов

После создания структуры решения были разработаны основные классы, представляющие сущности предметной области. Для работы с базой данных используется технология **Entity Framework Core**, которая обеспечивает удобное отображение объектов программы на таблицы базы данных.

Контекст базы данных

Для взаимодействия приложения с базой данных был создан класс **PinDbContext**, который наследуется от класса **DbContext**. Данный класс является центральным элементом работы с базой данных и отвечает за управление подключением, а также за взаимодействие с таблицами.

Внутри класса были объявлены наборы сущностей (**DbSet**), соответствующие таблицам базы данных:

- **PartnerTypes** — типы партнеров;
- **Partners** — партнеры;
- **Products** — товары;
- **PartnerProductSales** — продажи товаров партнерам.

```

using Ilin.PartnerApp.Lib.Entities;
using Microsoft.EntityFrameworkCore;

namespace Ilin.Partner.Lib.Context
{
    Ссылка: 26
    public class IlinDbContext : DbContext
    {
        Ссылка: 10
        public DbSet<PartnerType> PartnerTypes => Set<PartnerType>();
        Ссылка: 24 10/10 passing
        public DbSet<PartnerApp.Lib.Entities.Partner> Partners => Set<PartnerApp.Lib.Entities.Partner>();
        Ссылка: 15 6/6 passing
        public DbSet<Product> Products => Set<Product>();
        Ссылка: 15 4/4 passing
        public DbSet<PartnerProductSale> PartnerProductSales => Set<PartnerProductSale>();

        Ссылка: 4
        public IlinDbContext()
        {
        }

        Ссылка: 2
        public IlinDbContext(DbContextOptions<IlinDbContext> options)
            : base(options)
        {
        }

        Ссылка: 0
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (!optionsBuilder.IsConfigured)
            {
                optionsBuilder.UseNpgsql(
                    "Host=localhost;Port=5432;Database=ilin;Username=app;Password=123456789");
            }
        }

        Ссылка: 0
    }
}

```

Рисунок 4. Часть контекста подключения к базе данных

Контекст также содержит конфигурацию подключения к базе данных PostgreSQL, в которой указывается адрес сервера, имя базы данных и учетные данные пользователя приложения.

Использование контекста базы данных позволяет выполнять операции чтения, добавления, изменения и удаления данных с использованием объектно-ориентированной модели.

Класс **PartnerType**

Класс **PartnerType** представляет тип партнера и соответствует таблице **partner_types** в базе данных.

Данный класс содержит следующие основные поля:

- идентификатор типа партнера;
- название типа партнера.

```

namespace Ilin.PartnerApp.Lib.Entities
{
    [DataContract][Table("partner_types")]

    public class PartnerType
    {
        [DataMember]
        [Column("id")]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]

        public int Id { get; set; }

        [DataMember]
        [Column("name")]

        public string Name { get; set; } = string.Empty;

        public ICollection<Partner> Partners { get; set; } = new List<Partner>();
    }
}

```

Рисунок 5. Код класса PartnerType

Типы партнеров используются для классификации партнеров организации. Например, партнер может относиться к розничным магазинам, оптовым компаниям или дилерам.

Один тип партнера может быть связан с несколькими партнерами, что реализовано через навигационное свойство коллекции партнеров.

Класс Partner

Класс **Partner** представляет карточку партнера организации. Данный класс соответствует таблице **partners**.

В классе содержатся следующие основные данные:

- тип партнера;
- название компании;
- юридический адрес;
- ИНН;
- фамилия, имя и отчество директора;
- контактный телефон;
- адрес электронной почты;
- рейтинг партнера;
- путь к логотипу компании;

- места продаж.

Также в классе реализована связь с типом партнера и коллекцией продаж, что позволяет получать информацию о всех продажах, связанных с данным партнером.

Класс Product

Класс **Product** соответствует таблице **products** и предназначен для хранения информации о товарах, реализуемых партнерам.

В классе содержатся следующие поля:

- артикул товара;
- тип товара;
- название товара;
- цена товара;
- признак активности товара.

```
namespace Ilin.PartnerApp.Lib.Entities
{
    [DataContract]
    [Table("products")]
    Ссылка: 11
    public class Product
    {
        [DataMember]
        [Column("id")]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        Ссылка: 17 | 6/6 passing
        public int Id { get; set; }

        [DataMember]
        [Column("article")]
        Ссылка: 13 | 6/6 passing
        public string Article { get; set; } = string.Empty;

        [DataMember]
        [Column("product_type")]
        Ссылка: 5
        public string? ProductType { get; set; }

        [DataMember]
        [Column("name")]
        Ссылка: 16 | 6/6 passing
        public string Name { get; set; } = string.Empty;

        [DataMember]
        [Column("price")]
        Ссылка: 15 | 6/6 passing
        public decimal Price { get; set; }

        [DataMember]
        [Column("is_active")]
        Ссылка: 13 | 6/6 passing
        public bool IsActive { get; set; }

        Ссылка: 2
        public ICollection<PartnerProductSale> PartnerProductSales { get; set; } = new List<PartnerProductSale>();
    }
}
```

Рисунок 6. Код класса Product

Поле активности используется для определения возможности оформления продажи данного товара. Если товар помечен как неактивный, оформление новой продажи по нему запрещается.

Класс PartnerProductSale

Класс **PartnerProductSale** соответствует таблице **partner_product_sales** и предназначен для хранения информации о продажах товаров партнерам.

Каждая запись продажи содержит:

- идентификатор партнера;
- идентификатор товара;
- количество проданного товара;
- дату продажи;
- базовую цену товара на момент продажи;
- процент скидки партнера;
- цену товара с учетом скидки;
- комментарий к продаже.

Особенностью реализации является сохранение **базовой цены товара и процента скидки на момент оформления продажи**. Это позволяет сохранить историческую корректность данных даже в случае изменения цены товара в будущем.

4.3 Реализация бизнес-логики приложения

После реализации моделей данных и контекста базы данных была разработана бизнес-логика приложения. Для этого были созданы отдельные сервисные классы, которые выполняют операции работы с данными и содержат основные правила функционирования системы.

Использование сервисного слоя позволяет отделить пользовательский интерфейс от логики обработки данных. Вся работа с базой данных осуществляется через соответствующие сервисы, что упрощает поддержку и развитие приложения.

В системе были реализованы следующие основные сервисы:

- **PartnerService** — управление партнерами;
- **PartnerTypeService** — управление типами партнеров;
- **ProductService** — управление товарами;
- **SaleService** — управление продажами товаров партнерам.

Каждый сервис содержит методы для выполнения основных операций над данными: добавления, изменения, удаления и получения информации из базы данных.

Сервис управления партнерами

Класс **PartnerService** предназначен для выполнения операций работы с партнерами.

Данный сервис реализует следующие основные функции:

- получение списка партнеров из базы данных;
- добавление нового партнера;
- изменение информации о партнере;
- удаление партнера;
- проверка корректности вводимых данных.

При добавлении или изменении партнера выполняется проверка корректности данных. Например, система проверяет существование выбранного типа партнера, а также корректность значений некоторых полей.


```
public async Task<int> AddPartnerAsync(PartnerEditModel model)
{
    ValidatePartner(model);

    await EnsurePartnerTypeExistsAsync(model.TypeId);
    await EnsureUniquePartnerFieldsAsync(model);

    var partner = new Ilin.PartnerApp.Lib.Entities.Partner
    {
        TypeId = model.TypeId,
        CompanyName = model.CompanyName.Trim(),
        Address = model.Address.Trim(),
        Inn = NormalizeNullable(model.Inn),
        DirectorFullname = model.DirectorFullname.Trim(),
        Phone = model.Phone.Trim(),
        Email = model.Email.Trim(),
        Rating = model.Rating,
        LogoPath = NormalizeNullable(model.LogoPath),
        SalesPlaces = NormalizeNullable(model.SalesPlaces),
        CreatedAt = DateTime.UtcNow
    };

    _context.Partners.Add(partner);
    await _context.SaveChangesAsync();

    return partner.Id;
}
```

Рисунок 7. Метод добавления партнера

```

private static void ValidatePartner(PartnerEditModel model)
{
    if (model.TypeId <= 0)
        throw new ArgumentException("Не выбран тип партнера.");

    if (string.IsNullOrEmpty(model.CompanyName))
        throw new ArgumentException("Введите наименование компании.");

    if (string.IsNullOrEmpty(model.Address))
        throw new ArgumentException("Введите адрес.");

    if (string.IsNullOrEmpty(model.DirectorFullname))
        throw new ArgumentException("Введите ФИО директора.");

    if (string.IsNullOrEmpty(model.Phone))
        throw new ArgumentException("Введите телефон.");

    if (string.IsNullOrEmpty(model.Email))
        throw new ArgumentException("Введите email.");

    if (model.Rating < 0)
        throw new ArgumentException("Рейтинг не может быть отрицательным.");

    if (!string.IsNullOrEmpty(model.Inn))
    {
        var inn = model.Inn.Trim();

        if (inn.Length != 10 && inn.Length != 12)
            throw new ArgumentException("ИНН должен содержать 10 или 12 символов.");
    }
}

```

Рисунок 8. Метод проверки значений в полях

Также реализована проверка при удалении партнера. Если у партнера существует история продаж, его удаление запрещается. Это позволяет сохранить целостность данных системы.

```

public async Task DeletePartnerAsync(int partnerId)
{
    var partner = await _context.Partners
        .FirstOrDefaultAsync(x => x.Id == partnerId);

    if (partner == null)
        throw new InvalidOperationException("Партнер не найден.");

    var hasSales = await _context.PartnerProductSales
        .AnyAsync(x => x.PartnerId == partnerId);

    if (hasSales)
        throw new InvalidOperationException("Нельзя удалить партнера, у которого есть история продаж.");

    _context.Partners.Remove(partner);
    await _context.SaveChangesAsync();
}

```

Рисунок 9. Метод удаления партнера

Сервис управления типами партнеров

Класс **PartnerTypeService** используется для управления типами партнеров.

Основные функции данного сервиса включают:

- получение списка типов партнеров;
- добавление нового типа партнера;
- изменение названия типа партнера;
- удаление типа партнера.

Перед удалением типа партнера система проверяет, используется ли данный тип в таблице партнеров. Если существуют партнеры, связанные с данным типом, операция удаления запрещается.

```
public async Task DeletePartnerTypeAsync(int id)
{
    var entity = await _context.PartnerTypes
        .FirstOrDefaultAsync(x => x.Id == id);

    if (entity == null)
        throw new InvalidOperationException("Тип партнера не найден.");

    var isUsed = await _context.Partners
        .AsNoTracking()
        .AnyAsync(x => x.TypeId == id);

    if (isUsed)
        throw new InvalidOperationException("Нельзя удалить тип партнера, который используется в карточках партнеров.");

    _context.PartnerTypes.Remove(entity);
    await _context.SaveChangesAsync();
}

private static void Validate(PartnerTypeEditModel model)
{
    if (string.IsNullOrEmpty(model.Name))
        throw new ArgumentException("Введите название типа партнера.");
}
```

Рисунок 10. Метод удаления типа и метод проверки данных в поле имени
Это правило предотвращает нарушение ссылочной целостности данных.

Сервис управления товарами

Класс **ProductService** реализует операции управления товарами.

Основные функции сервиса:

- получение списка товаров;
- добавление нового товара;
- изменение информации о товаре;
- удаление товара;
- проверка корректности данных товара.

При добавлении или изменении товара выполняется проверка корректности цены. Цена товара не может иметь отрицательное значение.

Ссылка: 2

```
private static void Validate(ProductEditModel model)
{
    if (string.IsNullOrEmpty(model.Article))
        throw new ArgumentException("Введите артикул.");

    if (string.IsNullOrEmpty(model.Name))
        throw new ArgumentException("Введите название товара.");

    if (model.Price < 0)
        throw new ArgumentException("Цена не может быть отрицательной.");
}
```

Ссылка: 2

```
private async Task EnsureUniqueArticleAsync(ProductEditModel model)
{
    var article = model.Article.Trim();

    var exists = await _context.Products
        .AsNoTracking()
        .AnyAsync(x => x.Article == article && x.Id != model.Id);

    if (exists)
        throw new InvalidOperationException("Товар с таким артикулом уже существует.");
}
```

Рисунок 11. Проверка значений и уникальности артикула

Также реализована проверка при удалении товара. Если товар уже используется в продажах, его удаление запрещается.

Дополнительно в системе реализовано поле **IsActive**, которое определяет, доступен ли товар для продажи.

```

public async Task<int> AddProductAsync(ProductEditModel model)
{
    Validate(model);
    await EnsureUniqueArticleAsync(model);

    var entity = new Product
    {
        Article = model.Article.Trim(),
        ProductType = NormalizeNullable(model.ProductType),
        Name = model.Name.Trim(),
        Price = model.Price,
        IsActive = model.IsActive
    };

    _context.Products.Add(entity);
    await _context.SaveChangesAsync();

    return entity.Id;
}

```

Рисунок 12. Метод добавления с полем IsActive

Если товар помечен как неактивный, оформление новой продажи по данному товару невозможно.

Сервис управления продажами

Класс **SaleService** предназначен для регистрации и обработки продаж товаров партнерам.

Основные функции данного сервиса:

- получение списка продаж;
- оформление новой продажи;
- изменение информации о продаже;
- удаление записи продажи;
- расчет стоимости продажи.

Перед созданием продажи выполняется проверка существования партнера и выбранного товара. Также система проверяет, является ли выбранный товар активным.

```

public async Task<SaleCalculationResultDTO> CalculateSaleAsync(int partnerId, int productId, int quantity, int? editingSaleId = null)
{
    if (partnerId <= 0)
        throw new ArgumentException("Не выбран партнер.");

    if (productId <= 0)
        throw new ArgumentException("Не выбран товар.");

    if (quantity <= 0)
        throw new ArgumentException("Количество должно быть больше нуля.");

    var product = await _context.Products
        .AsNoTracking()
        .FirstOrDefaultAsync(x => x.Id == productId);

    if (product == null)
        throw new InvalidOperationException("Товар не найден.");

    if (!product.IsActive)
        throw new InvalidOperationException("Нельзя оформить продажу по неактивному товару.");

    var totalQuantity = await _context.PartnerProductSales
        .AsNoTracking()
        .Where(x => x.PartnerId == partnerId && (!editingSaleId.HasValue || x.Id != editingSaleId.Value))
        .SumAsync(x => (int?)x.Quantity) ?? 0;

    var discountPercent = PartnerDiscountCalculator.Calculate(totalQuantity);

    var unitPrice = Math.Round(product.Price * (1m - discountPercent / 100m), 2, MidpointRounding.AwayFromZero);
    var totalAmount = Math.Round(unitPrice * quantity, 2, MidpointRounding.AwayFromZero);

    return new SaleCalculationResultDTO
    {
        BasePrice = product.Price,
        DiscountPercent = discountPercent,
        UnitPrice = unitPrice,
        TotalAmount = totalAmount
    };
}

```

Рисунок 13. Проверка наличия партнера, товара и активности товара

Если товар помечен как неактивный, оформление продажи запрещается.

При оформлении продажи система автоматически выполняет расчет стоимости с учетом скидки партнера.

Алгоритм расчета скидки партнера

В системе реализован отдельный механизм определения скидки партнера. Размер скидки зависит от суммарного объема товаров, проданных данному партнеру за весь период сотрудничества.

Алгоритм расчета скидки реализован в отдельном классе **PartnerDiscountCalculator**.

```

namespace Ilin.PartnerApp.Lib.Services
{
    Ссылка: 9
    public static class PartnerDiscountCalculator
    {
        Ссылка: 9 7/7 passing
        public static int Calculate(int totalQuantity)
        {
            if (totalQuantity < 0)
            {
                throw new ArgumentOutOfRangeException(nameof(totalQuantity), "Количество не может быть отрицательным.");
            }

            if (totalQuantity < 10000)
            {
                return 0;
            }

            if (totalQuantity < 50000)
            {
                return 5;
            }

            if (totalQuantity < 300000)
            {
                return 10;
            }

            return 15;
        }
    }
}

```

Рисунок 14. Класс, отвечающий за расчет скидки партнера

Размер скидки определяется следующим образом:

- при суммарном объеме продаж менее 10 000 единиц — скидка отсутствует;
- при объеме продаж от 10 000 единиц — скидка составляет 5 %;
- при объеме продаж от 50 000 единиц — скидка составляет 10 %;
- при объеме продаж от 300 000 единиц — скидка составляет 15 %.

При создании новой продажи система вычисляет текущий суммарный объем продаж партнера, определяет соответствующий уровень скидки и применяет его к цене товара.

Сохранение исторической корректности данных

Особенностью реализации системы является сохранение ценовых параметров продажи на момент ее оформления.

При создании записи продажи в базе данных сохраняются следующие значения:

- базовая цена товара;
- процент скидки партнера;
- цена товара с учетом скидки.

Сохранение этих данных позволяет обеспечить историческую корректность информации. Даже если цена товара изменится в будущем, ранее

созданные продажи будут содержать исходные значения, действовавшие на момент оформления сделки.

Таким образом, система обеспечивает корректное хранение информации о продажах и исключает искажение данных при изменении цен товаров.

4.4 Написание модульных тестов

Для повышения надежности разработанного программного продукта и проверки корректности реализации бизнес-логики в системе был реализован механизм модульного тестирования.

Модульное тестирование позволяет проверить корректность работы отдельных компонентов программы независимо от пользовательского интерфейса и других частей системы. Такой подход дает возможность выявить ошибки на ранних этапах разработки и убедиться в правильности работы ключевых алгоритмов.

Для реализации модульных тестов был создан отдельный проект тестирования, использующий фреймворк **MSTest**, который является одним из стандартных средств тестирования приложений на платформе .NET.

Создание отдельного тестового проекта позволяет изолировать тестовый код от основной программы и выполнять автоматическую проверку работы отдельных методов и компонентов системы.

Создание тестового проекта

В среде разработки **Microsoft Visual Studio** был создан отдельный проект типа **MSTest Test Project**. Данный проект был добавлен в общее решение системы и связан с библиотекой классов, содержащей основную бизнес-логику приложения.

Тестовый проект содержит набор тестовых классов, каждый из которых предназначен для проверки определенной части функциональности системы.

Для удобства организации тестирования были созданы следующие тестовые классы:

- тесты для алгоритма расчета скидки партнера;

- тесты для сервиса работы с продажами;
- тесты для сервиса работы с партнерами.

Каждый тестовый класс содержит набор методов тестирования, помеченных специальным атрибутом **[TestMethod]**, который указывает системе тестирования, что данный метод является тестом и должен выполняться при запуске тестирования.

Использование тестовой базы данных

Поскольку основные сервисы системы работают с базой данных, для выполнения тестов необходимо было обеспечить возможность работы с данными без подключения к реальной базе PostgreSQL.

Для решения данной задачи был использован специальный провайдер **Entity Framework Core InMemory**, который позволяет создавать временную базу данных в оперативной памяти.

Использование InMemory базы данных имеет несколько преимуществ:

- тесты выполняются значительно быстрее;
- не требуется подключение к реальному серверу базы данных;
- данные тестов полностью изолированы и не влияют на рабочую базу данных;
- каждый тест может использовать отдельную временную базу данных.

При выполнении теста создается новый экземпляр контекста базы данных, который использует временную InMemory-базу. После завершения теста все данные автоматически удаляются.

Таким образом обеспечивается независимость тестов и отсутствие влияния одного теста на результаты другого.

Тестирование алгоритма расчета скидки

Одним из ключевых элементов бизнес-логики системы является алгоритм расчета скидки партнера. Для проверки корректности работы данного алгоритма были реализованы отдельные тесты для класса **PartnerDiscountCalculator**.

В тестах проверяются различные сценарии работы алгоритма, соответствующие правилам системы скидок.

Были реализованы тесты для следующих случаев:

- расчет скидки при объеме продаж менее 10 000 единиц;
- расчет скидки при достижении уровня 10 000 единиц;
- расчет скидки при достижении уровня 50 000 единиц;
- расчет скидки при достижении уровня 300 000 единиц;
- проверка корректной обработки некорректных входных данных.

```
[TestMethod]
[Ссылка: 0]
public void Calculate_ShouldReturn_0_WhenQuantityIsLessThan10000()
{
    var result = PartnerDiscountCalculator.Calculate(9999);

    Assert.AreEqual(0, result);
}

[TestMethod]
[Ссылка: 0]
public void Calculate_ShouldReturn_5_WhenQuantityIs10000()
{
    var result = PartnerDiscountCalculator.Calculate(10000);

    Assert.AreEqual(5, result);
}

[TestMethod]
[Ссылка: 0]
public void Calculate_ShouldReturn_5_WhenQuantityIs49999()
{
    var result = PartnerDiscountCalculator.Calculate(49999);

    Assert.AreEqual(5, result);
}
```

Рисунок 15. Часть методов тестирования расчета скидки

Каждый тест проверяет, что при заданном значении суммарного объема продаж алгоритм возвращает правильный процент скидки.

Это позволяет убедиться в том, что система корректно определяет уровень скидки партнера при оформлении продаж.

Тестирование сервиса продаж

Особое внимание было уделено тестированию сервиса **SaleService**, так как данный сервис содержит наиболее сложную бизнес-логику приложения.

В процессе тестирования были реализованы тесты, проверяющие следующие сценарии:

- корректный расчет стоимости продажи;
- применение скидки партнера;
- запрет оформления продажи по неактивному товару;
- корректное сохранение ценовых параметров продажи.

При выполнении теста в InMemory-базу данных добавляются тестовые записи партнеров и товаров, после чего выполняется вызов метода оформления продажи.

Тесты проверяют, что:

- базовая цена товара определяется корректно;
- скидка партнера рассчитывается в соответствии с правилами системы;
- итоговая цена продажи вычисляется правильно;
- итоговая сумма продажи соответствует количеству товара и цене с учетом скидки.

Также проверяется корректность обработки ошибок. Например, если товар помечен как неактивный, система должна запретить оформление продажи и выбросить соответствующее исключение.

```

[TestMethod]
// Ссылка: 1 | 1/1 passing
public async Task CalculateSaleAsync_ShouldThrowException_ForInactiveProduct()
{
    using var context = CreateContext(nameof(CalculateSaleAsync_ShouldThrowException_ForInactiveProduct));

    context.Partners.Add(new PartnerApp.Lib.Entities.Partner
    {
        Id = 1,
        TypeId = 1,
        CompanyName = "ООО Гамма",
        Address = "Адрес",
        DirectorFullname = "Директор",
        Phone = "123",
        Email = "c@test.com",
        Rating = 1,
        CreatedAt = DateTime.UtcNow
    });

    context.Products.Add(new Product
    {
        Id = 1,
        Article = "A-001",
        Name = "Товар 1",
        Price = 1000,
        IsActive = false
    });

    await context.SaveChangesAsync();

    var service = new SaleService(context);

    var ex = await Assert.ThrowsAsync<InvalidOperationException>(async () =>
    {
        await service.CalculateSaleAsync(1, 1, 1);
    });

    Assert.AreEqual("Нельзя оформить продажу по неактивному товару.", ex.Message);
}

```

Рисунок 16. Тест на создание покупки неактивного товара

Тестирование сервиса партнеров

Дополнительно были реализованы тесты для сервиса **PartnerService**, который отвечает за управление партнерами системы.

В данных тестах проверяются основные операции работы с партнерами:

- добавление нового партнера;
- изменение информации о партнере;
- удаление партнера;
- проверка ограничений системы.

В процессе тестирования проверяется, что система корректно сохраняет данные партнера в базе данных, а также выполняет необходимые проверки корректности данных.

Например, реализованы тесты, проверяющие запрет удаления партнера, если у него существует история продаж. Такая проверка позволяет сохранить целостность данных системы.

```
[TestMethod]
// Ссылка: 1 | 1/1 passing
public async Task DeletePartnerAsync_ShouldThrow_WhenPartnerHasSales()
{
    using var context = CreateContext(nameof(DeletePartnerAsync_ShouldThrow_WhenPartnerHasSales));
    var typeId = await AddPartnerTypeAsync(context);

    var partner = new PartnerApp.Lib.Entities.Partner
    {
        TypeId = typeId,
        CompanyName = "ООО С продажами",
        Address = "Адрес",
        Inn = "7708888888",
        DirectorFullname = "Директор",
        Phone = "123",
        Email = "sales@test.com",
        Rating = 1,
        CreatedAt = DateTime.UtcNow
    };

    var product = new Product
    {
        Article = "P-001",
        Name = "Товар 1",
        Price = 1000m,
        IsActive = true
    };

    context.Partners.Add(partner);
    context.Products.Add(product);
    await context.SaveChangesAsync();

    context.PartnerProductSales.Add(new PartnerProductSale
    {
        PartnerId = partner.Id,
        ProductId = product.Id,
        Quantity = 10,
        SaleDate = DateOnly.FromDateTime(DateTime.Today),
        BasePrice = 1000m,
        DiscountPercent = 0,
        UnitPrice = 1000m,
        Comment = null
    });

    await context.SaveChangesAsync();

    var service = new PartnerService(context);

    var ex = await Assert.ThrowsAsync<InvalidOperationException>(async () =>
    {
        await service.DeletePartnerAsync(partner.Id);
    });

    Assert.AreEqual("Нельзя удалить партнера, у которого есть история продаж.", ex.Message);
}
```

Рисунок 17. Тест на удаление партнера с историей продаж

Результаты тестирования

После написания тестов было выполнено их автоматическое выполнение с использованием встроенных средств тестирования Visual Studio.

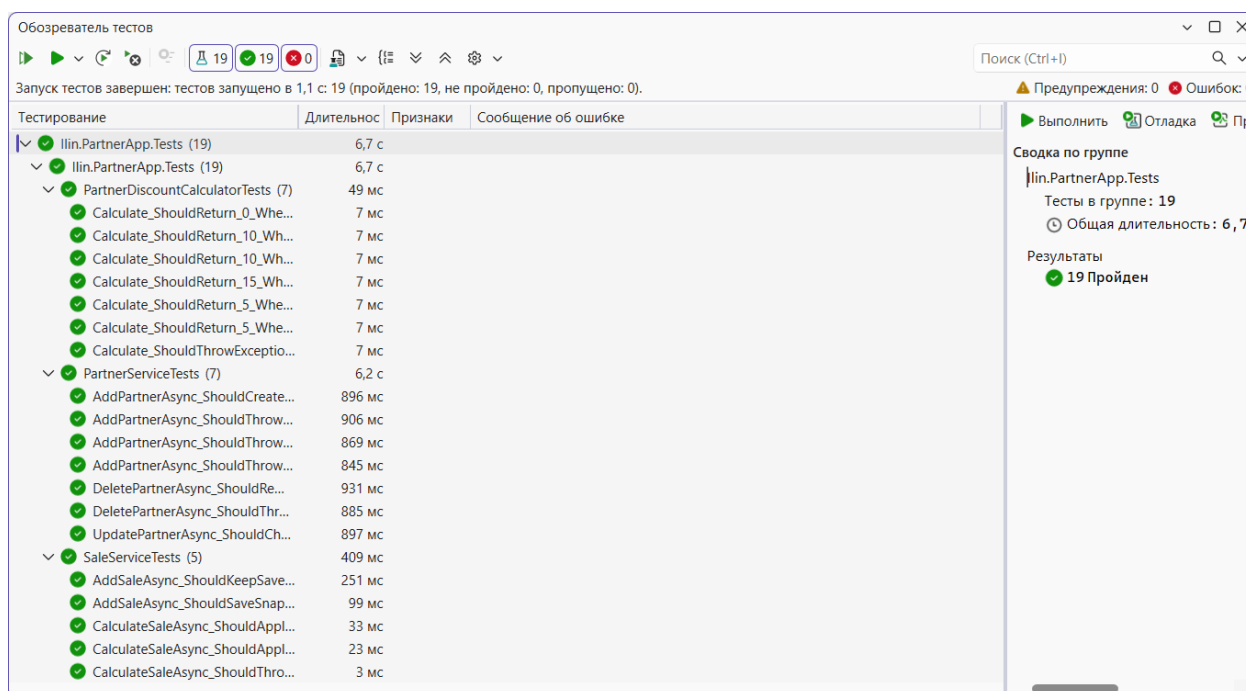


Рисунок 18. Результаты тестирования

Результаты тестирования показали корректную работу всех реализованных алгоритмов и сервисов. Все тесты были успешно выполнены, что подтверждает правильность реализации ключевых компонентов системы.

Использование модульного тестирования позволило убедиться в корректности бизнес-логики приложения, а также повысить надежность программного продукта.

Кроме того, наличие тестов упрощает дальнейшее развитие системы, так как позволяет автоматически проверять корректность работы программы после внесения изменений в код.

4.5 Реализация пользовательского интерфейса приложения

После разработки структуры базы данных и реализации бизнес-логики системы был выполнен этап создания пользовательского интерфейса приложения. Интерфейс программы был разработан с использованием технологии **WPF (Windows Presentation Foundation)**, входящей в состав платформы .NET.

WPF позволяет создавать современные настольные приложения с графическим интерфейсом, поддерживающим гибкую систему компоновки элементов, стилизацию компонентов и удобное взаимодействие пользователя с системой.

Основной задачей пользовательского интерфейса является обеспечение удобного доступа к данным системы, а также предоставление пользователю возможности выполнять операции добавления, изменения и удаления информации.

В интерфейсе программы реализовано несколько отдельных окон, каждое из которых отвечает за работу с определенной сущностью системы.

К основным элементам интерфейса относятся:

- главное окно приложения;
- окно управления типами партнеров;
- окно управления товарами;
- окно учета продаж;
- окна добавления и редактирования данных.

Каждое окно содержит элементы управления, позволяющие пользователю выполнять операции **создания, изменения, удаления и просмотра данных**.

Главное окно приложения

Главное окно является центральным элементом интерфейса системы. После запуска программы пользователю отображается список партнеров, зарегистрированных в базе данных.

В главном окне реализованы следующие элементы интерфейса:

- панель навигации;
- область отображения партнеров;
- панель управления данными;
- область отображения истории продаж выбранного партнера.

Панель навигации содержит кнопки перехода к другим разделам системы, таким как управление товарами, типами партнеров и продажами.

Область отображения партнеров представляет собой список карточек партнеров. Каждая карточка содержит основную информацию о партнере:

- название компании;
- тип партнера;
- контактные данные;
- рейтинг партнера;
- размер текущей скидки.

Использование карточек позволяет сделать отображение данных более наглядным и удобным для пользователя.

При выборе партнера в списке автоматически отображается история его продаж в нижней части окна. Это позволяет пользователю быстро просмотреть информацию о сотрудничестве с выбранным партнером.

Также в главном окне реализована панель управления, содержащая кнопки для выполнения основных операций:

- добавление нового партнера;
- редактирование информации о партнере;
- удаление партнера;
- обновление списка данных.

Перед удалением партнера система выполняет проверку наличия связанных продаж. Если у партнера существует история продаж, удаление записи запрещается.

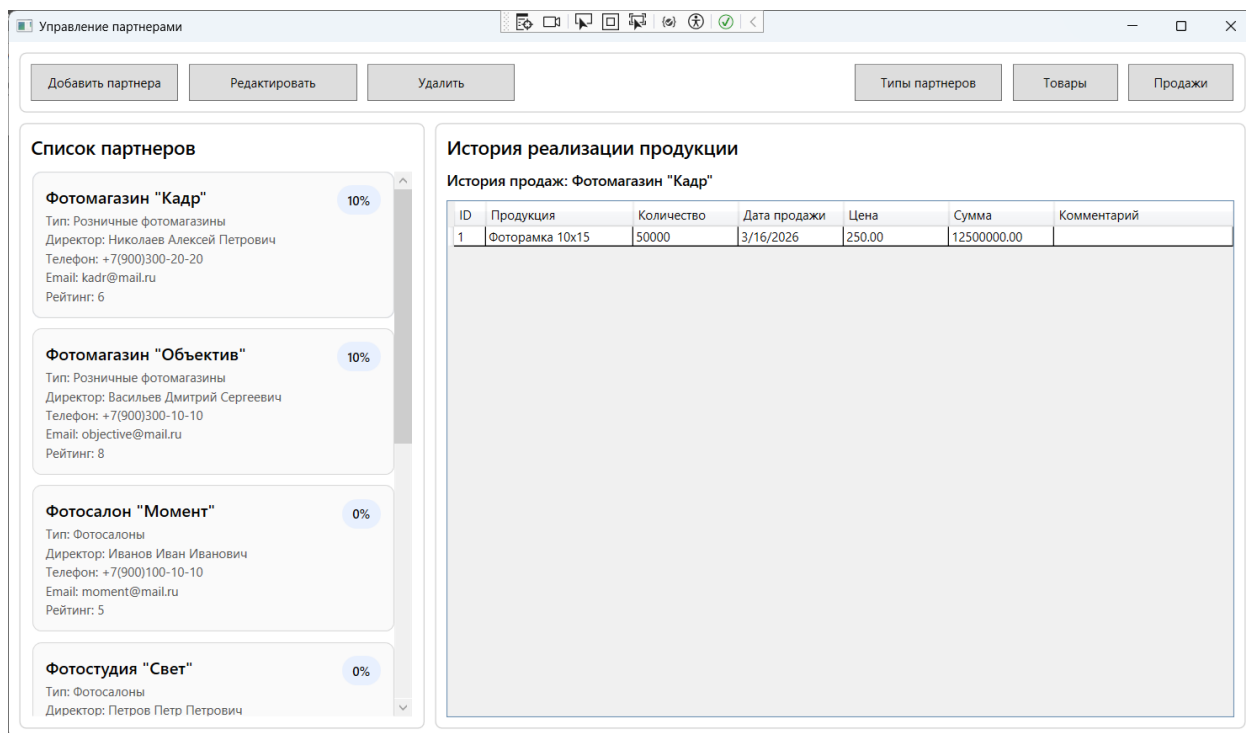


Рисунок 19. Главное окно приложения

Окно управления типами партнеров

Для управления типами партнеров в системе реализовано отдельное окно. Данное окно предназначено для добавления, изменения и удаления типов партнеров.

Тип партнера используется для классификации партнеров системы. Например, партнер может относиться к следующим категориям:

- розничный магазин;
- оптовый поставщик;
- дилерская компания.

Окно содержит таблицу, в которой отображаются все существующие типы партнеров.

В верхней части окна расположена панель управления, содержащая кнопки выполнения операций:

- добавление нового типа партнера;
- изменение названия типа партнера;
- удаление типа партнера;
- обновление списка типов партнеров.

При попытке удаления типа партнера система проверяет, используется ли данный тип в таблице партнеров. Если тип связан хотя бы с одним партнером, операция удаления запрещается.

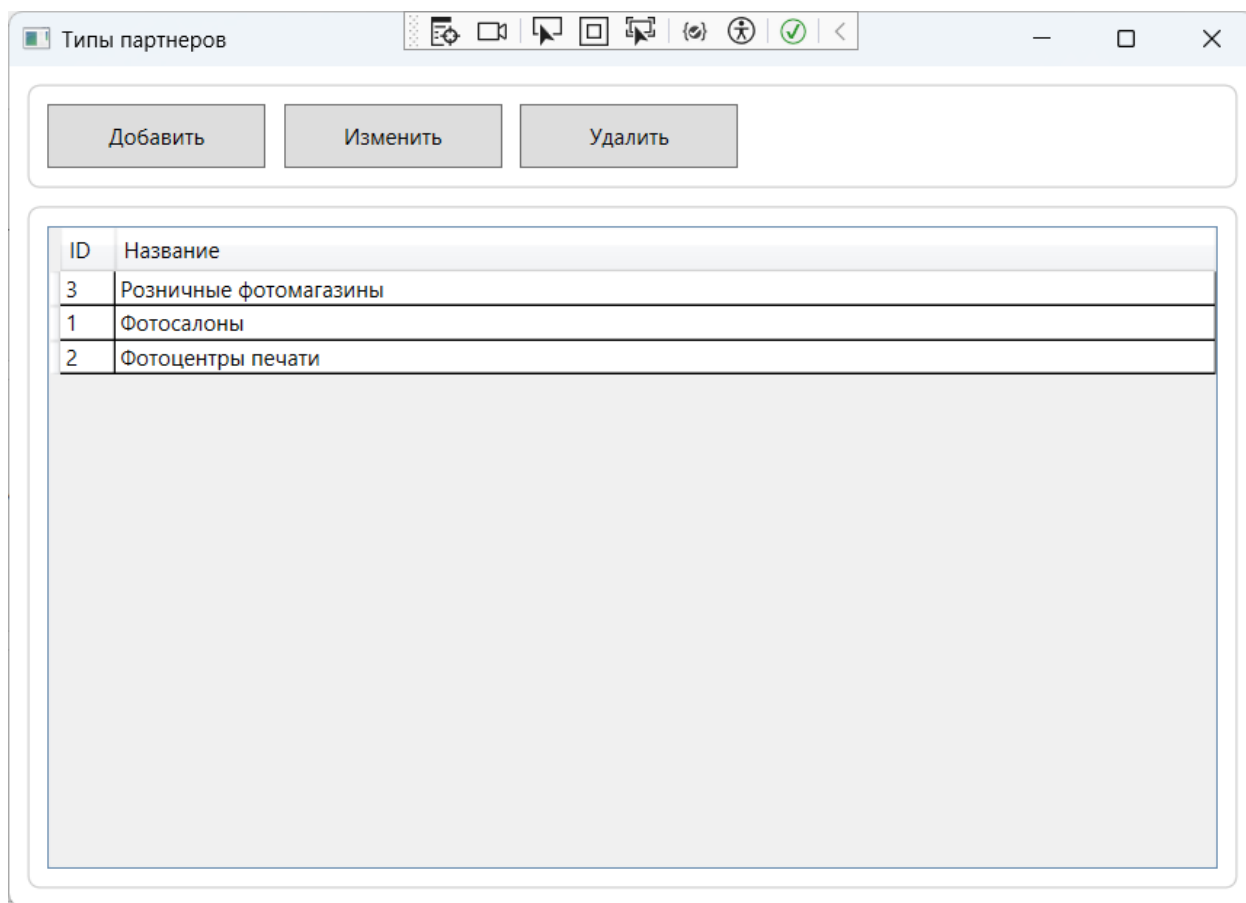


Рисунок 20. Окно управления типами партнеров

Окно управления товарами

Для работы с товарами в системе реализовано отдельное окно управления товарами. Данное окно позволяет пользователю просматривать список товаров, а также выполнять операции добавления, изменения и удаления товаров.

В таблице товаров отображается следующая информация:

- артикул товара;
- тип товара;
- название товара;
- цена товара;
- признак активности товара.

Признак активности товара используется для управления возможностью оформления продаж. Если товар помечен как неактивный, оформление новых продаж по данному товару запрещается.

В верхней части окна расположена панель управления, содержащая кнопки выполнения основных операций:

- добавление нового товара;
- изменение информации о товаре;
- удаление товара;
- обновление списка товаров.

Перед удалением товара система выполняет проверку наличия связанных продаж. Если товар уже используется в истории продаж, его удаление запрещается.

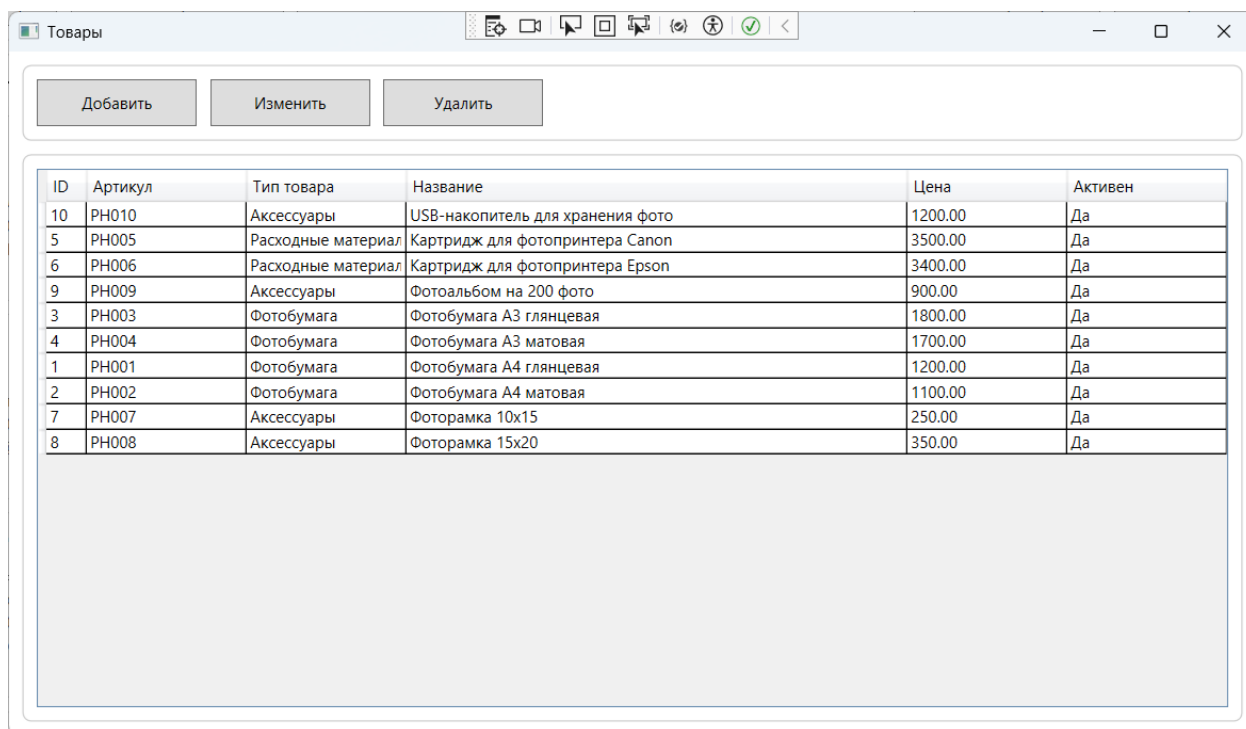


Рисунок 21. Окно управления товарами

Окно учета продаж

Для регистрации продаж товаров партнерам в системе реализовано окно учета продаж.

Данное окно содержит таблицу, в которой отображается список всех зарегистрированных продаж. Для каждой продажи отображается следующая информация:

- партнер, которому была выполнена продажа;
- товар;
- количество проданного товара;
- дата продажи;
- базовая цена товара;
- процент скидки партнера;
- цена товара с учетом скидки;
- итоговая сумма продажи;
- комментарий к продаже.

В верхней части окна расположена панель управления, позволяющая выполнять операции:

- добавление новой продажи;
- редактирование существующей продажи;
- удаление записи продажи;
- обновление списка продаж.

При добавлении новой продажи пользователь выбирает партнера, товар и указывает количество товара. После ввода данных система автоматически рассчитывает стоимость продажи с учетом скидки партнера.

Также выполняется проверка активности выбранного товара. Если товар помечен как неактивный, система запрещает оформление продажи.

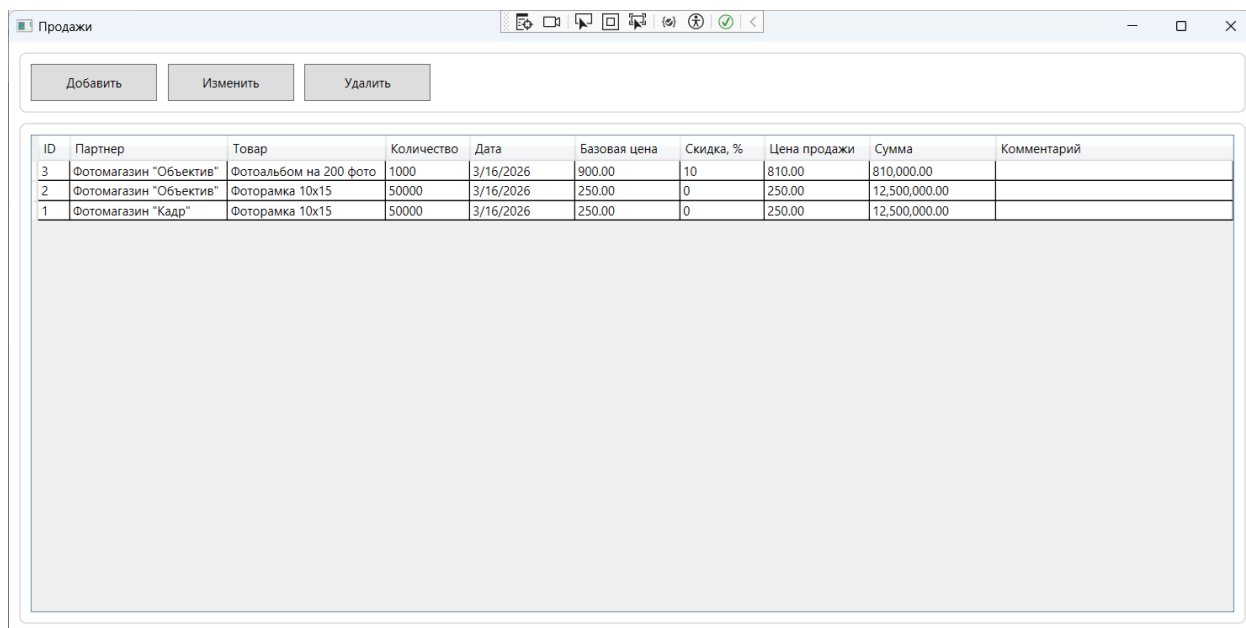


Рисунок 22. Окно управления продажами

Окна добавления и редактирования данных

Для добавления и редактирования информации в системе используются специальные диалоговые окна.

Такие окна используются для:

- добавления партнера;
- редактирования данных партнера;
- добавления товара;
- редактирования товара;
- добавления продажи;
- редактирования продажи.

Каждое окно содержит форму ввода данных, состоящую из различных элементов управления:

- текстовые поля;
- выпадающие списки;
- переключатели;
- поле выбора даты.

После заполнения формы пользователь может сохранить введенные данные. При этом система выполняет проверку корректности введенной информации.

Если данные введены некорректно, пользователю выводится сообщение об ошибке, после чего он может исправить введенные значения.

Использование отдельных окон редактирования позволяет сделать интерфейс программы более удобным и структурированным.

Рисунок 23. Окно создания новой продажи

Удобство работы с интерфейсом

При разработке интерфейса приложения особое внимание было уделено удобству работы пользователя с системой.

Основные принципы реализации интерфейса включают:

- разделение функциональности по отдельным окнам;
- использование таблиц для отображения данных;
- наличие панелей управления с основными операциями;
- автоматическое обновление отображаемых данных;
- проверку корректности вводимой информации.

Такой подход позволяет пользователю быстро ориентироваться в интерфейсе программы и эффективно выполнять необходимые операции.

5. ВЫВОД

В ходе выполнения данной работы была разработана программная система «Система работы с партнерами», предназначенная для автоматизации учета партнеров организации, управления товарами и регистрации продаж.

В процессе разработки были выполнены все основные этапы создания программного продукта: проектирование структуры базы данных, реализация моделей данных, разработка бизнес-логики системы, создание пользовательского интерфейса и проведение модульного тестирования.

Для хранения данных была использована система управления базами данных **PostgreSQL**, которая обеспечивает надежное и эффективное хранение информации. Была разработана структура базы данных, включающая таблицы типов партнеров, партнеров, товаров и продаж. Между таблицами были реализованы необходимые связи, обеспечивающие целостность данных.

Для взаимодействия приложения с базой данных использовалась технология **Entity Framework Core**, позволяющая работать с данными на уровне объектов и упрощающая реализацию операций чтения и изменения информации.

В рамках разработки были реализованы основные классы, представляющие сущности предметной области системы, а также создан контекст базы данных, обеспечивающий взаимодействие приложения с таблицами PostgreSQL.

Важной частью разработки стало создание сервисного слоя приложения, реализующего бизнес-логику системы. Были разработаны сервисы для работы с партнерами, типами партнеров, товарами и продажами. Эти сервисы выполняют операции добавления, изменения, удаления и получения данных, а также обеспечивают выполнение необходимых проверок и ограничений.

Особое внимание было уделено реализации механизма расчета скидок для партнеров. Система автоматически определяет размер скидки на основе

суммарного объема продаж партнера и применяет ее при оформлении новых продаж. При этом в базе данных сохраняются базовая цена товара и процент скидки на момент оформления продажи, что обеспечивает корректное хранение исторических данных.

Для повышения надежности системы были разработаны модульные тесты с использованием фреймворка **MSTest**. Тестирование позволило проверить корректность работы ключевых компонентов системы, включая алгоритм расчета скидки и сервисы обработки продаж и партнеров. Для выполнения тестов использовалась временная база данных **Entity Framework Core InMemory**, что позволило изолировать тестовую среду от основной базы данных.

Также был разработан графический пользовательский интерфейс приложения на основе технологии **WPF**. Интерфейс программы включает главное окно со списком партнеров, а также отдельные окна для управления типами партнеров, товарами и продажами. Пользователь может выполнять основные операции работы с данными через удобные формы и таблицы.

Разработанная система позволяет автоматизировать учет партнеров и продаж, обеспечивает удобный доступ к информации и упрощает управление данными организации. Реализованный программный продукт может использоваться в качестве основы для дальнейшего развития и расширения функциональности системы.